ARC UI · UIARC.DEV · PROOF OF CONCEPT

Flagship Dashboard

Northwind: a motion-heavy SaaS analytics product built with Arc UI

Generated 2026-11-11

A motion-heavy SaaS analytics dashboard built with Vite, React 19, and TypeScript to show what an AI agent can produce with the Arc UI component library.

Overview

Northwind is the "flagship product demo" proof of concept for Arc UI, a React UI library positioned as *"the React UI library your AI agents can build with."* It is a self-contained, realistic-feeling analytics product: a revenue overview with animated charts, a KPI row with count-up numbers and sparklines, a Kanban board, a sortable transactions table, a ⌘K command palette, toasts, light/dark theming with live accent switching, and a navigation sidebar.

Why it exists

The POC tests a single thesis: **an AI agent, working from Arc's components and semantic design tokens, can assemble a real, polished, accessible application without a bespoke design system or a chart library.** Specifically, it demonstrates that:

- Arc's components compose into a full product surface (shell, navigation, data views, overlays).
- Theming is driven entirely by semantic CSS custom properties, so one token change recolors the whole app, including charts.
- Motion is part of the component contract, and every animation has a `prefers-reduced-motion` fallback.
- Charts can be built in plain SVG plus the `motion` package, with no third-party charting dependency.

Features

Overview view

- **KPI row** of four `StatCard`s: Monthly recurring revenue, Active users, Conversion rate, and Net revenue retention. Each shows an animated value, a delta chip, and a sparkline.

- **Revenue** `AreaChart` with a 6M / 12M / YTD range switch (`Tabs`). Switching range re-keys the chart so it redraws.
- **Plan-mix** `DonutChart` (Enterprise, Business, Pro, Free) with a "Accounts" center label and an interactive legend.
- **Channel** `BarChart` (Organic, Referral, Paid, Social, Email, Direct) with compact value formatting.
- **Activity feed** with avatars, badges, and relative timestamps.

Projects view

- **Kanban board** titled "Q4 Roadmap" with four columns: Backlog, In Progress, Review, and Done.
- Each task shows a colored tag, a priority badge, and assignee avatars.
- Moving a card uses the chevron controls on the card. Each move animates the card across columns through a shared `layoutId` transition and fires a toast. Moving a card into **Done** shows a "Task completed" success toast.
- A "New task" button shows an informational toast only; no task composer exists yet.

Transactions view

- Summary tiles for total paid and pending count, using `AnimatedNumber` .
- A status filter (`Tabs`) that narrows the table.
- A sortable `DataTable` with columns for ID, customer (with `Avatar`), plan, status (`Badge` , tone mapped per status), and amount.
- An **Export** button that shows a toast confirming a CSV will be emailed. No file is generated.

Settings view

- **Appearance card:** a theme control (`ThemeToggle`) and six accent swatches (Violet, Blue, Green, Amber, Coral, Rose).
- **Notifications card:** four `Switch` controls (product, billing, weekly, security). Their state is local to the page and is not persisted.
- **Danger zone:** a "Delete workspace" action that opens a confirmation `Dialog` . Confirming shows a danger toast and does not delete anything.

Global shell

- **Sidebar** with Workspace and Account sections. Projects shows a badge of 8. The active item has a shared-layout highlight. The sidebar footer shows a placeholder user.
- **Mobile drawer:** below 900px, the sidebar is replaced by a slide-in drawer opened from a menu button in the top bar.
- **Top bar:** page title and subtitle that animate on route change, a search button that opens the command palette, a notifications button that shows an info toast, the theme toggle, and a "New report" button that shows a success toast.

Command palette (⌘K / Ctrl+K)

Opened from the keyboard shortcut or the top-bar search button. Commands are grouped as follows:

Group	Commands
Navigate	Go to Overview, Projects, Transactions, Settings
Actions	Switch to light/dark theme, Create a report, Export transactions
Accent	Set accent to each of the six accent colors

Filtering is substring-based over labels and keywords. Keyboard navigation and a gliding highlight are built into the `CommandPalette` component.

Theming

- Light and dark themes are switched by setting `data-theme` on `<html>`.
- The initial theme follows the saved `localStorage` value (`arc-theme`), falling back to `prefers-color-scheme`. An inline script in `index.html` applies it before first paint to avoid a flash.
- The theme toggle, the command palette, and the Settings page all call the same `useTheme()` hook.

Accent colors

- Six accents are selectable in the UI: **violet** (default), blue, green, amber, coral, and rose.
- The accent is stored as `data-accent` on `<html>` and persisted to `localStorage` under `arc-accent`.
- `foundation.css` also defines `neutral` and `orange` accent selectors. They are not exposed in the UI.

Reduced motion

Every component that animates has a reduced-motion path, implemented with motion's `useReducedMotion()` hook or `@media (prefers-reduced-motion: reduce)` rules in CSS modules:

- `AnimatedNumber` snaps to its final value.
- Charts appear without drawing.
- Page and drawer transitions fall back to short opacity fades.
- Toast exit animations become opacity-only.
- Button, Switch, Dialog, and Tabs disable or shorten transitions.

Keyboard navigation works in the command palette, the data table, tabs, and dialogs.

Architecture and Tech Stack

Layer	Choice	Notes
Build	Vite 8 with `@vitejs/plugin-react` 6	Path alias `@` maps to `src/`.
UI runtime	React 19 (`react`, `react-dom` ^19.3)	`StrictMode` enabled in `src/main.tsx`.
Language	TypeScript (`~7.0.2`)	`tsconfig.app.json` enables `noUnusedLocals`, `noUnusedParameters`, and `erasableSyntaxOnly`. `pnpm build` runs `tsc -b` before `vite build`.
Styling	CSS Modules over Arc's `foundation.css` semantic tokens	No CSS framework and no utility-class library.
Animation	`motion` ^14 (`motion/react`)	Used for entrances, springs, shared layout, and count-up tweens.
Charts	Hand-built SVG	`AreaChart`, `BarChart`, `DonutChart`, and `Sparkline`, with path helpers in `chart-utils.ts`. No chart library.
Primitives	Radix UI (`@radix-ui/react-dialog`, `-switch`, `-tabs`)	Used by the reused `Dialog`, `Switch`, and `Tabs` components.
Icons	`lucide-react`	
Lint	oxlint	Rules: `react/rules-of-hooks` and `react/only-export-components`.

Theming and tokens

- `src/components/arc/foundation.css` defines semantic tokens such as `--background`, `--foreground`, `--surface-muted`, `--accent`, `--accent-strong`, `--accent-subtle`, `--accent-foreground`, `--info`, `--warning`, `--success`, and duration/easing variables. Dark mode is applied under `[data-theme="dark"]`, and each accent under `[data-accent="…"]`.
- Components reference only these tokens, so theme and accent changes are pure CSS. There is no per-component theme logic.
- `src/index.css` sets global resets, theme-aware scrollbars, and maps the Arc font tokens to a system font stack, so no web font is loaded.

Motion system

- `src/components/arc/motion-tokens.ts` defines the shared timing system: durations (`instant` through `considered`), easings (`enter`, `exit`, `standard`, `inOut`), springs (`responsive`, `gentle`, `snappy`, `smooth`, `morph`), stagger steps, and blur radii.
- The `smooth`, `snappy`, and `morph` springs are critically damped (`bounce: 0`), so elements settle without overshoot.

- `src/components/arc/lib/motion-tokens.ts` re-exports the same object for import-path convenience.

State and persistence

- **Theme and accent:** `src/lib/use-theme.ts` reads the initial values from `<html>` (set by the inline script in `index.html`) and writes changes back to `<html>` and to `localStorage` (`arc-theme`, `arc-accent`). Storage errors are caught and ignored.
- **View routing:** there is no router. `App.tsx` holds the current view in `useState` and renders one page at a time.
- **Data:** all content comes from the static `src/data.ts`. There is no API or persistence for tasks, transactions, or settings.

Application composition

```
main.tsx
 └─ App (ToastProvider)
     └─ Dashboard
         ├─ Sidebar (desktop and mobile drawer)
         ├─ Topbar (title, search, theme toggle, notifications, New report)
         ├─ Page: OverviewPage | ProjectsPage | TransactionsPage | SettingsPage
         └─ CommandPalette (⌘K)
```

Component Inventory

Components in `src/components/arc/`. **Radix** means built directly on a Radix UI primitive. **New** means built for this demo with no underlying primitive library.

Component	Folder	Status	Description
`Button`	`button/`	Radix	Variants `primary`, `secondary`, `ghost`, `danger`; sizes `sm`, `md`, `lg`; label morph, press spring, and loading state.
`Switch`	`switch/`	Radix	Radix-based toggle with a stretchy spring thumb.
`Dialog`	`dialog/`	Radix	Radix-based modal with spring entrance and a text-swap title. Exports `Dialog`, `DialogTrigger`, `DialogClose`, and `DialogContent`.
`Tabs`	`tabs/`	Radix	Radix-based tabs with a gliding selection highlight and directional panel transitions. Exports `Tabs`, `TabsList`, `TabsTrigger`, and `TabsContent`.
`Card`	`card/`	New	Staggered rise-in entrance and hover state. Exports `Card`, `CardHeader`, `CardTitle`, `CardDescription`, `CardContent`, `CardFooter`, and `CardEyebrow`.
`Badge`	`badge/`	New	Thirteen semantic tones (`neutral`, `teal`, `sky`, `orchid`, `violet`, `rose`, `amber`, `green`, `success`, `warning`, `danger`, `info`, `accent`), with `dot` and `solid` options.
`Avatar`, `AvatarGroup`, `AvatarMore`	`avatar/`	New	Gradient initials, overlapping stacks, a `+N` overflow chip, and a presence `status` indicator.
`AnimatedNumber`	`number/`	New	Eased count-up from the previous value, with a custom formatter. Snaps under reduced motion.
`Sparkline`	`chart/`	New	Self-drawing SVG trend line with gradient fill and end dot.
`AreaChart`	`chart/`	New	Responsive multi-series area chart with path draw-in, hover crosshair, and a glass tooltip.
`BarChart`	`chart/`	New	Bars that spring up from the baseline with stagger. Hover dims other bars and shows a value label.
`DonutChart`	`chart/`	New	Arcs that sweep in via `stroke-dashoffset`, an interactive legend, and a center readout.
`StatCard`	`stat-card/`	New	KPI tile combining `AnimatedNumber`, a delta chip, and `Sparkline`.
`Kanban`	`kanban/`	New	Board with cards that glide between columns via shared-layout animation. Moves are made with per-card chevron controls.
`DataTable`	`table/`	New	Generic `DataTable<T>` with sortable headers, animated row reordering, and a staggered entrance.

Component	Folder	Status	Description
`CommandPalette`	`command-palette/`	New	⌘K menu with substring filtering, full keyboard navigation, and a gliding highlight.
`ToastProvider`, `useToast`	`toast/`	New	Portal-based toast stack with context hook, stacked layout animation, and a progress bar. Default duration 4200 ms.
`ThemeToggle`	`theme-toggle/`	New	Sun/moon icon with a spin-and-scale crossfade.
`Sidebar`	`sidebar/`	New	Navigation with sections, a shared-layout active indicator, badges, and a user footer.

Totals: 19 component modules, of which 4 are built on Radix UI primitives and 15 are new. Each module is a `.tsx` file with a sibling `.module.css` file, except `sparkline`, `animated-number`, and the `chart-utils.ts` helper, which have no CSS module.

Supporting (non-component) modules

File	Purpose
`foundation.css`	Arc semantic design tokens: theme, accent, typography, radius, and motion variables.
`motion-tokens.ts`	Shared durations, easings, springs, stagger, and blur values.
`lib/motion-tokens.ts`	Re-export of `motion-tokens.ts`.
`chart/chart-utils.ts`	Point, line, smooth-path, and area-path helpers for the SVG charts.

Project Structure

Excludes `node_modules/` and `dist/`.

```
flagship-dashboard/
├── README.md                # Project overview, component list, run instructions
├── package.json              # Scripts (dev, build, lint, preview) and dependencies
├── pnpm-lock.yaml             # Locked dependency versions for pnpm
├── index.html                # Entry HTML; inline script applies theme/accent before paint
├── vite.config.ts             # React plugin and "@" → src alias
├── tsconfig.json              # Root TypeScript project references
├── tsconfig.app.json           # App compile settings (src/, strict unused checks)
├── tsconfig.node.json            # Config for Vite/Node tooling files
├── .oxlintrc.json             # oxlint configuration (React hook and export rules)
├── components.json             # shadcn-style registry config (base-nova, lucide)
├── .gitignore                # Ignored paths
├── public/
│   └── favicon.svg            # Browser tab icon
├── screenshots/               # Six reference PNGs (see Screenshots)
└── src/
    ├── main.tsx               # Entry: imports foundation.css + index.css, renders <App />
    ├── index.css              # App globals: font tokens, resets, theme-aware scrollbars
    ├── App.tsx                # Shell: sidebar, drawer, top bar, routing, ⌘K command list
    ├── App.module.css          # Layout/page styles, shared by App and all four pages
    ├── data.ts                # Static mock data: KPIs, revenue, channels, Kanban, etc.
    ├── lib/
    │   ├── use-theme.ts         # Theme/accent state; syncs to <html> and localStorage
    │   └── utils.ts             # cn() class joiner, clamp(), number formatters
    ├── pages/
    │   ├── overview.tsx         # KPIs, revenue area chart, plan donut, channel bars
    │   ├── projects.tsx         # "Q4 Roadmap" Kanban board
    │   ├── transactions.tsx      # Summary tiles, status filter, sortable table, export
    │   └── settings.tsx         # Appearance, notification switches, delete dialog
    └── components/arc/
        ├── foundation.css        # Arc semantic tokens (light/dark, accent selectors)
        ├── motion-tokens.ts       # Shared motion tokens
        ├── lib/motion-tokens.ts    # Re-export of motion tokens
        ├── avatar/               # Avatar, AvatarGroup, AvatarMore
        ├── badge/                # Badge with 13 tones
        ├── button/               # Button (Radix)
        ├── card/                 # Card and its parts
        ├── chart/                # AreaChart, BarChart, DonutChart, Sparkline
        ├── command-palette/       # ⌘K command palette
        ├── dialog/               # Dialog (Radix)
        ├── kanban/               # Kanban board
        ├── number/               # AnimatedNumber
        ├── sidebar/              # Sidebar navigation
        ├── stat-card/            # StatCard
        ├── switch/               # Switch (Radix)
        ├── table/                # DataTable
        ├── tabs/                 # Tabs (Radix)
        ├── theme-toggle/          # ThemeToggle
        └── toast/                # ToastProvider and useToast
```

Getting Started

Prerequisites

- Node.js 20 or later (the project was developed on Node 24).
- pnpm (developed on pnpm 11). npm also works, per the README.

Install

```
pnpm install
```

Run the development server

```
pnpm dev
```

Opens at http://localhost:5173.

Build for production

```
pnpm build      # runs `tsc -b` (type check), then `vite build` into dist/
```

Preview the production build

```
pnpm preview
```

Lint

```
pnpm lint       # runs oxlint
```

Screenshots

Captured with headless Chrome and stored in `screenshots/`.

File	Caption
`01-overview-light.png`	Overview dashboard in the light theme: KPI row, revenue area chart, plan-mix donut, and channel bars.
`02-overview-dark.png`	The same Overview dashboard in the dark theme.
`03-projects-dark.png`	Projects view in the dark theme: the "Q4 Roadmap" Kanban board.
`04-transactions-light.png`	Transactions view in the light theme: summary tiles and the sortable data table.
`05-settings-light.png`	Settings view in the light theme: appearance controls with the accent picker.
`06-command-palette-dark.png`	The ⌘K command palette open over the dark theme.

Notes and Limitations

- **Placeholder data.** All content in `src/data.ts` is mock data. Transactions, tasks, KPIs, and activity are static and cannot be created, edited, or persisted.
- **Placeholder profile.** The sidebar user footer is hard-coded in `App.tsx` as a placeholder name and email address. Replace these before reusing the shell for anything else.
- **Non-functional actions.** The "New report", "New task", "Export", "Notifications", and "Delete workspace" actions only display toasts. Delete does not remove any data. Notification switches on the Settings page are local state and are not saved.
- **Kanban interaction.** The `Kanban` component's doc comment says a card can be dragged onto another column, but no drag handlers exist in the implementation. Moves happen through the per-card chevron buttons only. Either implement drag or correct the comment.
- **Unused accent values.** `foundation.css` defines `neutral` and `orange` accent selectors that the UI does not expose.
- **No routing.** Views are switched with component state. Deep links and browser back/forward do not work.
- **Single JavaScript bundle.** All four pages are imported eagerly in `App.tsx`, and there is no code splitting. Vite may therefore report a large-chunk warning on `pnpm build`. The README states the build passes cleanly. This document did not re-run the build to confirm the warning.
- **Unused directives.** Several files begin with a `"use client"` directive, which is a Next.js convention and has no effect in this Vite app.
- **Mobile layout.** The sidebar becomes a drawer below 900px and the Kanban stacks below 860px. Layouts were not exhaustively tested at every breakpoint.

- **Shared stylesheet.** The four page modules import `App.module.css` directly rather than owning their own styles, so changes to that file affect several views.